SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0603 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks:25**

Annexure A

Experiments

Code of Conduct:

I Mohammed Mudhassir A/192424367 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Mudhassir

1. Naïve Bayes Classifier

Aim: To implement Naïve Bayes classification and study feature independence and Laplace smoothing.

Experiment:

- Use a real-world dataset such as the **Iris** or **Student Placement** dataset.
- Split the data into training and testing sets.
- Train a Naïve Bayes classifier using different feature sets.
- Calculate **accuracy, precision, and recall**.
- Compare results with and without Laplace smoothing.

Analysis:
Naïve Bayes assumes that all features are conditionally independent given the class. In real datasets, features may be correlated, so this assumption can reduce performance. Laplace smoothing assigns a small non-zero probability to unseen feature values and prevents zero-probability errors.

Conclusion: Naïve Bayes is simple and effective, but its performance depends on the quality of probability estimation and the independence assumption.

---

2. Maximum Likelihood Estimation vs Bayesian Estimation

Aim: To estimate distribution parameters using MLE and Bayesian estimation.

Experiment:

- Consider normally distributed data.
- Calculate the **sample mean and variance** using MLE.
- Apply Bayesian estimation by introducing prior information about the mean.
- Repeat the experiment with **small and large sample sizes**.

Analysis:
MLE estimates parameters only from observed data. Bayesian estimation combines observed data with prior knowledge.

For a normal distribution:

$$\hat{\mu}_{MLE} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

As the sample size increases, the influence of the prior decreases and Bayesian estimates become closer to MLE.

Conclusion: MLE is strongly data-dependent, while Bayesian estimation is useful when the sample size is small or prior knowledge is available.

---

3. Expectation-Maximization Algorithm

Aim: To study the convergence behavior of the EM algorithm for mixture models.

Experiment:

- Use a dataset containing observations from multiple groups.
- Initialize cluster means and variances randomly.
- Perform the **E-step** to calculate membership probabilities.
- Perform the **M-step** to update model parameters.
- Repeat until convergence.
- Change initialization and number of clusters.

Analysis:
Different initializations can lead to different final likelihood values because EM can converge to a **local optimum**. Increasing the number of clusters can also increase computational complexity.

Strategies for improvement:

- Use multiple random initializations.
- Use K-Means initialization.
- Set a suitable convergence tolerance.
- Normalize the data.

Conclusion: EM generally increases likelihood at every iteration, but the final solution depends on initialization and may be a local optimum.

---

4. Bayesian Belief Network

Aim: To construct a BBN and study inference under different evidence conditions.

Example network:

$$Flu \rightarrow Fever$$
$$Flu \rightarrow Cough$$
$$Flu \rightarrow BodyPain$$

Experiment:

- Assign prior probability to Flu.

- Define conditional probabilities for Fever, Cough, and Body Pain.
- Calculate the probability of Flu.
- Add evidence such as Fever = Yes and observe the posterior probability.
- Modify conditional probabilities and repeat.

Analysis:
Evidence about dependent variables changes the posterior probability of their parent variable. Stronger dependencies produce larger changes in posterior probabilities.

Conclusion: BBNs provide a systematic way to represent dependencies and reason under uncertainty.

---

5. Candidate Elimination Algorithm

Aim: To study concept learning and the effect of hypothesis-space size.

Experiment:

- Use a dataset containing attributes and a target concept.
- Initialize the **Specific boundary (S)** and **General boundary (G)**.
- Process positive and negative examples.
- Update S and G after every example.
- Increase the number of attributes and training examples.

Analysis:
A larger hypothesis space provides more possible hypotheses but requires more training examples to identify the correct concept. A finite hypothesis space can be searched systematically, while an infinite hypothesis space may require stronger assumptions or restrictions.

The **mistake bound** represents the maximum number of prediction mistakes before the learner identifies a consistent hypothesis.

Conclusion: Hypothesis-space size directly affects learning complexity, sample requirements, and generalization.

---

6. Decision Tree Classifier

Aim: To compare entropy and Gini index splitting criteria.

Experiment:

- Use a real-world classification dataset.
- Split it into training and testing data.
- Build trees using **Entropy** and **Gini Index**.

- Measure accuracy and F1-score.
- Change maximum tree depth.
- Apply pruning and compare results.

Analysis:
Entropy measures information uncertainty, while Gini measures node impurity. A very deep tree may memorize training data and cause **overfitting**. Limiting depth or pruning improves generalization.

Conclusion: Proper tree depth and pruning can produce better test performance and reduce overfitting.

---

7. K-Nearest Neighbors

Aim: To study the effect of K, distance metrics, and feature scaling.

Experiment:

- Select a classification dataset.
- Normalize/standardize the features.
- Train KNN with different values such as **K = 3, 5, 7**.
- Compare **Euclidean and Manhattan** distances.
- Calculate accuracy and confusion matrix.
- Compare scaled and unscaled data.

Analysis:
A small K can be sensitive to noise, while a large K can oversmooth the decision boundary. Feature scaling is important because KNN is distance-based.

Conclusion: The choice of K, distance metric, and feature scaling significantly affects KNN performance.

---

8. Support Vector Machine

Aim: To compare different SVM kernels and hyperparameters.

Experiment:

- Use a binary classification dataset.
- Train SVM using:
    - Linear kernel
    - Polynomial kernel
    - RBF kernel

- Evaluate accuracy and precision.
- Vary **C** and **gamma**.

Analysis:
The linear kernel works well for linearly separable data. Polynomial and RBF kernels can model nonlinear boundaries. A high C tries to classify training samples correctly, while gamma controls the influence of individual samples in an RBF kernel.

Conclusion: Kernel selection and hyperparameter tuning strongly influence the SVM decision boundary and classification performance.

---

9. K-Means Clustering

Aim: To study the effect of cluster number, initialization, and feature scaling.

Experiment:

- Use an unlabeled dataset.
- Apply K-Means for different values of K.
- Record **inertia** and **silhouette score**.
- Compare different initialization methods.
- Repeat with and without feature scaling.

Analysis:
Inertia generally decreases as K increases. The silhouette score helps identify well-separated clusters. Poorly scaled features can dominate the distance calculation.

Conclusion: The best K should provide low inertia and a relatively high silhouette score while maintaining meaningful clusters.

---

10. Logistic Regression with Regularization

Aim: To study L1 and L2 regularization in binary classification.

Experiment:

- Train logistic regression without regularization.
- Train models using **L1** and **L2** regularization.
- Evaluate accuracy, precision, and recall.
- Compare the learned coefficients.

Analysis:
L1 regularization can force some coefficients to zero, making the model useful for feature selection. L2 reduces coefficient magnitude without usually making them exactly zero. Regularization reduces overfitting.

Conclusion: Regularization improves generalization and controls model complexity, especially when many features are present.

---

11. Random Forest Classifier

Aim: To study the effect of ensemble size and tree depth.

Experiment:

- Use a real-world classification dataset.
- Train Random Forest models with different numbers of trees such as 50, 100, and 200.
- Vary maximum depth.
- Calculate accuracy, precision, recall, and F1-score.
- Compare with a single Decision Tree.

Analysis:
Increasing the number of trees generally makes predictions more stable and reduces variance. Feature randomness makes individual trees different, which improves ensemble diversity and generalization.

Conclusion: Random Forest usually provides better stability and generalization than a single decision tree.

---

12. Principal Component Analysis

Aim: To reduce dimensionality using PCA and study its effect on classification.

Experiment:

- Select a high-dimensional dataset.
- Standardize the features.
- Apply PCA.
- Retain different numbers of components, such as 2, 5, and 10.
- Measure explained variance and classification accuracy.
- Compare computational time.

Analysis:
PCA transforms correlated features into principal components. More components retain more information but provide less dimensionality reduction. Too few components may cause information loss and reduce classification performance.

Conclusion: PCA can reduce computational cost while retaining most important information if an appropriate number of components is selected.

---

13. Gradient Descent

Aim: To study the effect of learning rate and initialization on convergence.

Experiment:

- Define a simple cost function.
- Initialize parameters with different values.
- Run Gradient Descent using learning rates such as **0.001, 0.01, and 0.1**.
- Record the number of iterations and cost values.

Analysis:
A very small learning rate causes slow convergence. A very large learning rate can cause oscillation or divergence. A suitable learning rate provides faster and stable convergence.

Improvements:

- Momentum
- Adaptive learning rates
- Adam optimizer
- Proper parameter initialization

Conclusion: Learning rate and initialization have a major effect on Gradient Descent convergence.

---

14. Neural Network Classifier

Aim: To study how neural network architecture affects classification performance.

Experiment:

- Select a classification dataset.
- Build neural networks with different:
    - Number of hidden layers
    - Number of neurons
    - Activation functions
- Train each model.
- Record accuracy and loss.
- Plot training and validation loss curves.

Analysis:
Adding layers and neurons increases the model's ability to learn complex patterns. However,

an excessively large network can overfit. Activation functions such as ReLU help neural networks learn nonlinear relationships.

Conclusion: A suitable network architecture provides a balance between learning capacity, convergence speed, and generalization.

---

15. Hierarchical Clustering

Aim: To compare different linkage methods in hierarchical clustering.

Experiment:

- Use an unlabeled dataset.
- Apply hierarchical clustering using:
    - Single linkage
    - Complete linkage
    - Average linkage
- Generate dendrograms.
- Calculate silhouette scores.
- Compare the resulting clusters.

Analysis:
Single linkage can produce long, chain-like clusters and is sensitive to noise. **Complete linkage** tends to create compact clusters. **Average linkage** provides a balance between the two.

The dendrogram shows how clusters are progressively merged, while the silhouette score measures cluster quality.

Conclusion: The linkage method significantly affects cluster formation. The best method depends on the structure and separation of the dataset.